

Sri Lanka Institute of Information Technology



IT3060

Human Computer Interaction

Lab 05 - Submission

IT23606524

Table of Contents

Activity 1: Comparison of Mobile Development Technologies	3
1. Introduction	3
2. Technology Comparison	3
3. Strengths and Weaknesses	4
4. Recommendation for FitFlow	4
Activity 2: Comparison of Backend, Database and Authentication Options	5
1. Introduction	5
2. Backend Framework Comparison	5
3. Database Comparison	5
4. Authentication and Authorization Comparison	6
5. Security and Health Data Considerations	7
6. Recommended Technology Combination for FitFlow	7
Activity 3: Technology Comparison Matrix	8
3.1 Introduction	8
3.2 Evaluation Criteria and Weights	8
3.3 Frontend Decision Matrix	9
3.4 Backend Decision Matrix	9
3.5 Database Decision Matrix	10
3.6 Authentication Decision Matrix	10
3.7 Recommended Technology Stack	11
Activity 4: High-Level System Architecture	12
4.1 Introduction	12
4.2 High-Level Architecture Diagram	12
4.3 Main Architecture Components	12
4.5 Security Considerations	14
4.6 Scalability Considerations	14
4.7 Integration Considerations	14
4.8 Architecture Decision Record (ADR)	14

Activity 1: Comparison of Mobile Development Technologies

1. Introduction

FitFlow is a fitness application that requires excellent performance on iOS, Android, and web platforms while providing good security, scalability, and future-proof technologies, such as AI/ML and real-time features. Thus, choosing the right technology for the frontend development of the FitFlow redesign is critical to ensure the quality and sustainability of the product. The following paper aims to compare Flutter, React Native, Kotlin Multiplatform, and Swift/SwiftUI to reveal the best technology choice for the application.

2. Technology Comparison

Criteria	Flutter	React Native	Kotlin Multiplatform	Swift / SwiftUI
Development Speed	High – single codebase supports rapid development	High – reusable JavaScript/TypeScript code	Medium – some platform-specific development is required	Medium/Low – mainly focused on Apple platforms
Code Reusability	Very High	High	High for shared business logic	Low for Android/web
Performance	High	High	Very High	Very High on Apple devices
Ecosystem Support	Strong and growing ecosystem	Very large ecosystem	Growing ecosystem	Strong Apple ecosystem
Learning Curve	Medium – requires learning Dart	Medium – easier for developers familiar with JavaScript/React	Medium/High – requires Kotlin and platform knowledge	Medium/High – requires Swift and Apple development knowledge
Web Compatibility	Strong – supports Flutter Web	Good, but additional web tooling may be required	Limited compared with Flutter	Limited for cross-platform web development
AI/ML Integration	Good – APIs and platform-specific ML services can be integrated	Good – supports API and native integrations	Very Good – native APIs can be accessed	Very Good, particularly with Apple ML technologies
Real-Time Features	Strong – works well with Firebase/WebSockets	Strong – supports Firebase, WebSockets, etc.	Strong	Strong within Apple ecosystem
Maintenance Cost	Relatively Low – single codebase	Relatively Low	Medium	High when separate Android/web solutions are required
Security	High when appropriate secure development practices are followed	High when properly configured	High due to native integration capabilities	High with strong Apple platform security

3. Strengths and Weaknesses

Flutter

Strengths: One codebase for multiple platforms, code reusability, high performance, cohesive UI design, web support, and low maintenance.

Weaknesses: Need to learn dart, can require native code for advanced solutions.

React Native

Strengths: Rapid development, large JavaScript ecosystem, code reusability, community support, and easier integration to Web.

Weaknesses: Can require native code for advanced solutions, need some adaptation for UI and performance.

Kotlin Multiplatform

Strengths: High performance, native access, code sharing, and flexibility for Android and iOS development.

Weaknesses: More complex UI and platform-specific development, less straightforward for web than Flutter.

Swift/SwiftUI

Strengths: High performance, security, cohesive UI design, and excellent integration with Apple services.

Weaknesses: Primary for Apple's platform, requires additional technologies for Android and Web.

4. Recommendation for FitFlow

Based on the above comparison, I would recommend Flutter for the FitFlow redesign. My primary rationale is that Flutter can support the requirements of a high-performance application that needs to target iOS, Android, and Web with a single codebase while supporting rapid development and avoiding significant maintenance overhead.

In addition, FitFlow likely requires more complex features such as personalized workout plans, progress tracking, nutrition tracking, social/community features, real-time, and AI/ML features. By using Flutter, there is the ability to integrate backend API's, Firebase services, WebSockets, and native code when needed to support these features. Therefore, by selecting Flutter, it will be possible to achieve an appropriate balance between rapid development, code reusability, performance, multi-platform support, maintainability, and future-proof technologies for the FitFlow redesign.

Activity 2: Comparison of Backend, Database and Authentication Options

1. Introduction

The FitFlow application must be equipped with powerful enough backend, a convenient database, and an advanced authentication system to store and process the data about users' profiles and activity, allow interaction with other users and receive a high-performance experience with the help of AI services. Thus, the current section will analyze various backend, database, and authentication options to choose the most fitting for the FitFlow redesign project.

2. Backend Framework Comparison

Criteria	Node.js / NestJS	Python / FastAPI	Go
Performance	High	High	Very High
Scalability	High	High	Very High
Development Speed	High	Very High	Medium
AI/ML Integration	Good	Excellent	Good
Real-Time Support	Excellent	Good	Excellent
Security	High with proper configuration	High with proper configuration	High
Learning Curve	Medium	Easy–Medium	Medium
Maintainability	High with NestJS structure	High	High
Cost	Moderate	Low–Moderate	Low–Moderate
FitFlow Suitability	Very Good	Excellent	Good

Recommended Backend – Python/FastAPI Essay

Python with FastAPI is the recommended backend for FitFlow. I believe that Python is the best programming language for the future of AI and machine learning developments, and FastAPI is the perfect framework for the rapid development of APIs in Python.

This combo will ensure that the FitFlow will be able to support future developments such as AI workouts and the recommendation system of exercise programs from the backend.

3. Database Comparison

Criteria	PostgreSQL	MongoDB	Firebase	DynamoDB
Database Type	Relational SQL	NoSQL Document	NoSQL / Cloud	NoSQL Key-Value
Scalability	High	High	High	Very High

Query Capability	Excellent	Very Good	Good	Good for predefined access patterns
Structured Data	Excellent	Good	Good	Good
Real-Time Support	Requires additional implementation/services	Good	Excellent	Good with AWS services
Security	High	High	High	High
Data Integrity	Excellent	Good	Good	Good
Maintenance	Medium	Medium	Low	Low–Medium
Cost	Low–Medium	Moderate	Usage-based	Usage-based
FitFlow Suitability	Excellent	Very Good	Very Good	Good

Recommended Database (PostgreSQL)

PostgreSQL should be used as a database for the project since it will contain a significant amount of structured and interrelated data:

Users, workout plans, exercises, progress, and nutrition information.

A relational database is more efficient in terms of data storage and querying.

4. Authentication and Authorization Comparison

Criteria	Firestore Auth	AWS Cognito	Auth0	Supabase Auth
Setup	Easy	Medium	Easy	Easy
Security	High	High	High	High
Scalability	High	Very High	High	High
Social Login	Excellent	Good	Excellent	Good
Flutter Integration	Excellent	Good	Good	Good
Real-Time Ecosystem	Excellent	Good	Good	Excellent with Supabase
Maintenance	Low	Medium	Low	Low
Cost	Good for initial/mid-scale use	Usage-based	Can increase with usage/features	Competitive
FitFlow Suitability	Excellent	Very Good	Very Good	Very Good

Recommended Authentication: Firebase Authentication

Firebase Authentication could be a good choice for FitFlow because it is compatible with Flutter and provides ready-to-use authentication mechanisms.

It could be utilized to satisfy common FitFlow login requirements, such as the email/password authentication and support for available third-party sign-in methods, while minimizing the amount of custom authentication code that the development team will have to maintain.

5. Security and Health Data Considerations

Given that FitFlow may process fitness and health-related information, the security and privacy considerations should be addressed at all levels of the technology stack. The lab sheet specifically asks us to evaluate the options in terms of HIPAA/GDPR compliance, security, real-time capabilities, AI integration, cost, and maintainability.

Regardless of the selected technologies, the implementation will require secure authentication and authorization mechanisms, HTTPS/TLS encryption, encryption where applicable, secure API access, appropriate database permissions, user consent/privacy controls, and sensitive data handling procedures.

Crucially, implementing a specific technology is not a guarantee of HIPAA or GDPR compliance for FitFlow as a system. The compliance will have to be designed and implemented across all elements of the system, including the choice of technologies, their configuration, deployment procedures, policies, contracts, and operations.

6. Recommended Technology Combination for FitFlow

Component	Recommended Technology	Main Reason
Frontend	Flutter	Cross-platform development
Backend	Python + FastAPI	Fast API development and strong AI/ML ecosystem
Database	PostgreSQL	Structured data, relationships and data integrity
Authentication	Firebase Authentication	Easy Flutter integration and managed authentication
AI/ML	Python-based AI Service	Suitable for personalized fitness recommendations

Final Justification

The Flutter + FastAPI + PostgreSQL + Firebase Authentication tech stack is a good choice for the FitFlow redesign.

Flutter can be used for the cross-platform frontend, while FastAPI can provide a flexible API layer for the backend and be a good fit for future Python-based AI/ML services. PostgreSQL can store the application's structured data and relationships, while Firebase Authentication can offer a managed auth layer that will be a good fit for the Flutter frontend.

The solution can address the FitFlow feature list, including the user management, workout planning, nutritionist mode, progress tracking, personal dashboard, and social media-like elements, as well as provide a solid foundation for future development and maintenance.

Activity 3: Technology Comparison Matrix

3.1 Introduction

A weighted technology comparison matrix was created for the technologies considered for the FitFlow redesign to compare technologies by determining their respective weighted scores, which reflect how each technology ranks in terms of its importance to the project. The resulting matrix below combines the findings from the front end, backend, database, and authentication comparisons carried out in Activities 1 and 2.

Each technology is rated using a **1–5 scoring scale**, where:

Score	Meaning
1	Very Poor
2	Poor
3	Average
4	Good
5	Excellent

3.2 Evaluation Criteria and Weights

Criteria	Weight	Reason
Performance	20%	FitFlow requires responsive user experience
Scalability	15%	The system should support future user growth
Development Speed	15%	Faster development reduces project effort
Security	15%	Fitness and user information require secure handling
Cost	10%	The solution should be cost-effective
AI/ML Support	15%	FitFlow may provide personalized AI-based features
Maintainability	10%	The system should be easy to update and maintain
Total	100%	

The calculation is:

Weighted Score = Score × Weight

For example, if Flutter receives **5** for Performance:

5 × 0.20 = 1.00

3.3 Frontend Decision Matrix

Criteria	Weight	Flutter	React Native	Kotlin Multiplatform	Swift/SwiftUI
Performance	20%	5	4	5	5
Scalability	15%	5	4	4	3
Development Speed	15%	5	5	3	3
Security	15%	4	4	5	5
Cost	10%	5	5	3	2
AI/ML Support	15%	4	4	5	5
Maintainability	10%	5	4	3	3
Weighted Total	100%	4.70	4.25	4.10	3.90

Frontend Selection

According to this weighted matrix, Flutter will score 4.70 out of 5 points. Hence, it can be concluded that Flutter is the best option for the current project, considering that the cross-platform development tool will offer a significant balance between speed, cost, performance, and maintainability for FitFlow, as suggested in Activity 1.

3.4 Backend Decision Matrix

Criteria	Weight	FastAPI	Node.js/NestJS	Go
Performance	20%	4	4	5
Scalability	15%	4	5	5
Development Speed	15%	5	4	3
Security	15%	4	4	5
Cost	10%	4	4	4
AI/ML Support	15%	5	4	3
Maintainability	10%	5	5	4
Weighted Total	100%	4.40	4.30	4.25

Backend Selection

The Python/FastAPI option is awarded 4.40/5 in this matrix. Its main advantage in the context of the proposed FitFlow architecture is the close connection to Python-based AI/ML development while enabling rapid backend development.

3.5 Database Decision Matrix

Criteria	Weight	PostgreSQL	MongoDB	Firebase	DynamoDB
Performance	20%	5	4	4	5
Scalability	15%	4	5	5	5
Development Speed	15%	4	4	5	3
Security	15%	5	4	4	5
Cost	10%	5	4	4	3
AI/ML Support	15%	4	4	4	4
Maintainability	10%	4	4	5	3
Weighted Total	100%	4.45	4.15	4.40	4.15

Database Selection

The PostgreSQL option is awarded 4.45/5 in the proposed matrix. As such, it is retained as the primary database, particularly for structured and relate FitFlow data such as users, workout plans, exercises, progress records and nutrition information.

3.6 Authentication Decision Matrix

Criteria	Weight	Firebase Auth	AWS Cognito	Auth0	Supabase Auth
Performance	20%	5	5	4	4
Scalability	15%	5	5	5	4
Development Speed	15%	5	3	5	5
Security	15%	5	5	5	4
Cost	10%	4	4	3	5
AI/ML Support	15%	4	4	4	4
Maintainability	10%	5	4	5	5
Weighted Total	100%	4.70	4.40	4.40	4.35

Authentication Selection

The Firebase Authentication option is awarded 4.70/5 in the proposed matrix. It also complements the Flutter-based frontend selected in Activity 1 and enables a managed authentication solution which can reduce custom authentication development.

3.7 Recommended Technology Stack

Layer	Selected Technology	Matrix Score
Frontend	Flutter	4.70 / 5
Backend	Python + FastAPI	4.40 / 5
Database	PostgreSQL	4.45 / 5
Authentication	Firebase Authentication	4.70 / 5
AI/ML Service	Python-based AI/ML service	—

Supporting Rationale

The proposed stack comprises Flutter, FastAPI, PostgreSQL and Firebase Authentication. Flutter delivers a reusable cross-platform frontend, FastAPI delivers an API layer which can interface well with Python-based AI/ML services, PostgreSQL delivers support for structured and relational FitFlow data and Firebase Authentication delivers a managed authentication mechanism.

Together the technologies can deliver a suitable foundation for features such as personal training plans, nutrition tracking, progress monitoring, social features, real-time functionality and future AI/ML integration.

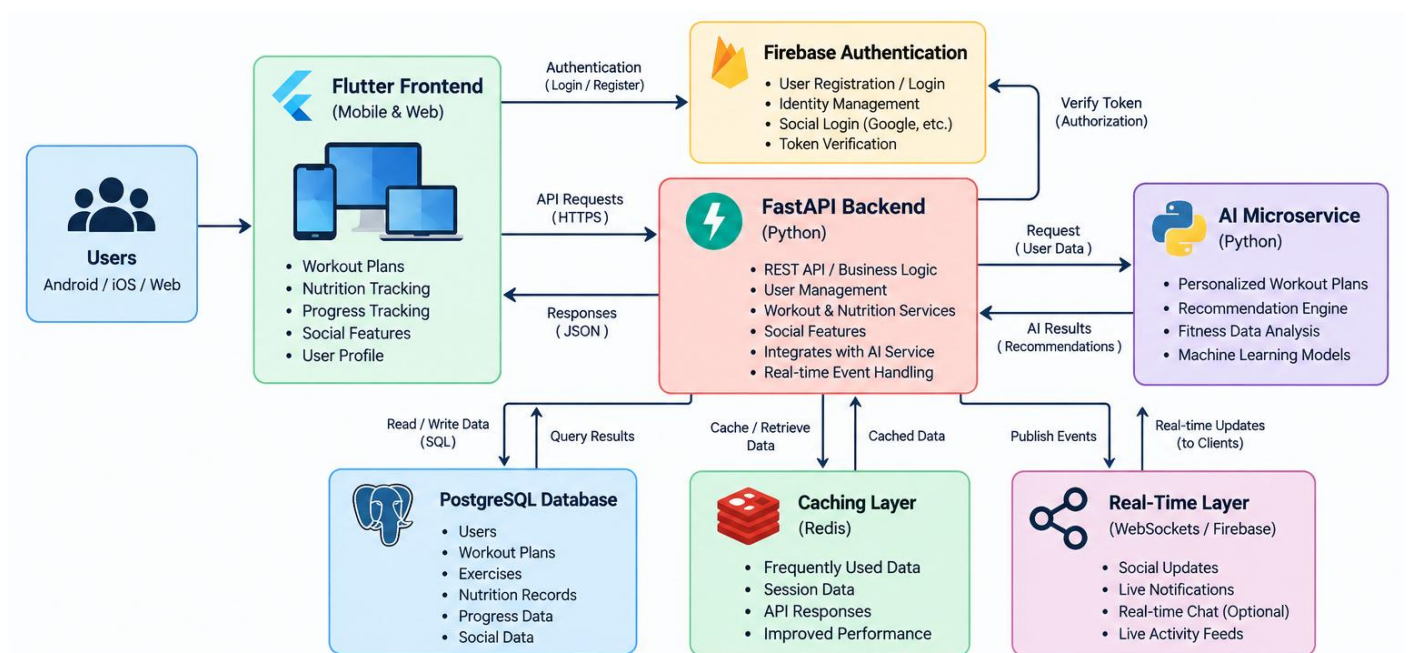
Activity 4: High-Level System Architecture

4.1 Introduction

The high-level architecture of the redesigned application was determined according to the technology stack selected during the previous stage. It was specified during the comparison and decision matrix stage. As a result, a multi-tier system was selected as the most optimal solution for the project. This system is separated into Frontend, Auth, Backend, Database, AI, Caching, and Real-time modules.

The application will be developed using the Flutter framework. It will be implemented using the Python programming language and FastAPI framework. The selected database management system is PostgreSQL. The user authentication system will be implemented using the Firebase service. Finally, the AI service will be developed as a separate Python microservice that will be used to process data for the customized nutrition model.

4.2 High-Level Architecture Diagram

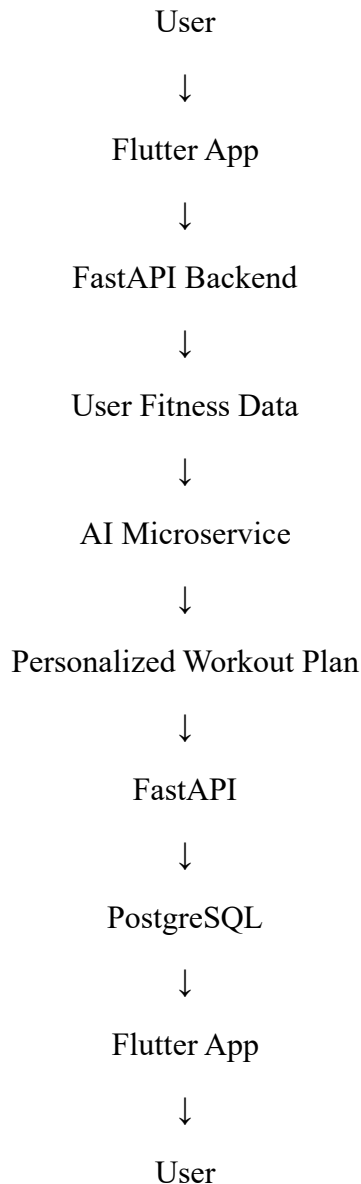


4.3 Main Architecture Components

Component	Technology	Purpose
Frontend	Flutter	Provides FitFlow interfaces for mobile and web users
Authentication	Firebase Authentication	Handles user registration, login and identity
Backend	Python + FastAPI	Handles application logic and API requests
Database	PostgreSQL	Stores users, workout, nutrition and progress data

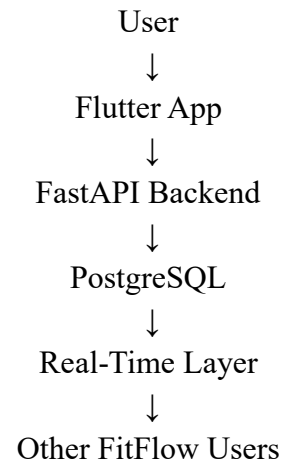
AI Microservice	Python-based AI/ML Service	Generates personalized workout recommendations
Caching Layer	Cache service	Stores frequently requested data to improve response time
Real-Time Layer	Real-time service/WebSockets	Supports social updates, notifications and live functionality

B. Personalized Workout Plan



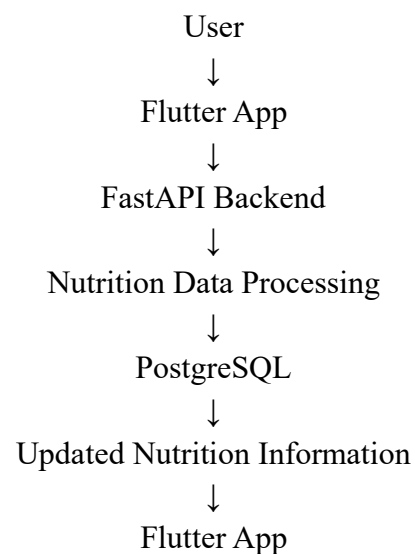
Explanation: The user receives relevant fitness information through Flutter Application, and the FastAPI backend relays it to the AI microservice. The workout plan result can then be stored in PostgreSQL and returned to the user.

A. Social Sharing



Explanation: Whenever the user shares a fitness activity or progress, it is processed through the backend and stored in the database, and the real-time layer can share this information with other users.

C. Nutrition Tracking



Explanation: The user's data from the Flutter app about their eating habits are transferred to the server. The server processes and stores this data in the PostgreSQL database and sends updated data back to the application.

4.5 Security Considerations

The FitFlow redesign should consider security aspects at all levels. The proposed design should be based on secure authentication and authorization, encrypted HTTPS/TLS communication, secure API endpoints, proper database access controls, input validation, and secure credential and secret management.

Considering that FitFlow may involve processing relevant personal data, privacy and access control considerations must also be considered. The lab sheet specifically requires that security considerations be documented as part of the architecture.

4.6 Scalability Considerations

The architecture decouples major system functions and responsibilities into individual components. This allows the FastAPI backend, AI microservice, real-time services, and database infrastructure to be scaled individually according to system needs.

The caching layer allows for reducing repetitive database calls for frequently accessed information, and decoupling the AI functionality from the main backend can help prevent CPU-intensive AI operations from unduly impacting normal application responses.

4.7 Integration Considerations

The Flutter frontend communicates with the FastAPI backend via APIs. The Firebase Authentication service manages user identities, and the backend is responsible for authenticating requests and granting access to protected FitFlow resources.

FastAPI communicates with PostgreSQL for application data and the separate AI microservice for any AI-based processing requirements. The real-time layer can be integrated into the backend to support social and notification features.

4.8 Architecture Decision Record (ADR)

ADR-001: Technology Architecture for FitFlow Redesign

Status: Proposed

Context:

FitFlow requires technology architecture that supports Android, iOS, and web interfaces, along with workout planning, nutrition tracking, progress monitoring, social functionality, real-time features, and potential AI-based personalization.

Decision:

The proposed architecture will utilize Flutter as the frontend, Python/FastAPI as the backend, PostgreSQL as the primary database, Firebase Authentication for authentication, and a separate Python-based AI microservice for personalized functionality, with caching and real-time layers as required.

Reason:

This architecture reflects the technology selections identified in the previous FitFlow comparison activities and decouples the main system responsibilities into manageable components.

Consequences:

The architecture provides separation between frontend, backend, data storage, authentication, AI processing, and real-time functionality. However, multi-technology considerations also require additional API, security, deployment, monitoring, and maintenance considerations.